

Thread Safety Quiz

These are exercises to practise thinking about shared state in multithreaded programs.

Increment a Global Counter

Multiple threads call the `Increment` function simultaneously. It updates a global counter `G`.

```
G←0

▽ {r}←Increment n;i;tmp
  :For i :In n
    tmp←G      A Take a snapshot. A thread may switch here.
    G←tmp+1    A Update state. Our snapshot might be out of date.
  :EndFor
  r←0
▽
```

Increment by 200,000 four times. The result should be 800,000.

```
G←0 ♦ Increment`4p2e5 ♦ G
800000
G←0 ♦ □TSYNC Increment&`4p2e5 ♦ G
771436
```

Where Dyalog Switches Threads

The interpreter may switch between threads:

- between any two lines of a defined function or operator
- on entry to a dfn or dop
- while waiting for □DL, for □DQ/□SR/□ED input, or for an external (□NA/.NET/OLE) call
- at the session prompt, □: or □

Nowhere else. In particular, it will **not** switch part-way through a single line of primitives, even with diamonds ♦.

Exercises

For each of the following examples, determine whether the scenario described is safe or unsafe. Variables in ALL-CAPS are shared global variables.

Answer with:

1. **SAFE** or **UNSAFE**
2. Name of the hazards, one or more of:
 - lost update
 - torn read
 - order violation
 - deadlock
 - livelock
 - starvation
 - none
3. Suggest a small fix that makes an unsafe scenario safer.

Unless a question says otherwise, assume more than one thread is running the code shown and that the globals already hold sensible values.

1 Rain Gauge

Each rain gauge adds its measurement to a running total by calling **AddRain**. The gauges report independently and none of them waits for the others.

```
▽ AddRain mm
  RAINFALL ← RAINFALL + mm
▽
```

2 Temperature Sensor

Temperature sensors append to a central log. One of the endpoints accepts readings in Fahrenheit and converts the reading before adding it to the log.

```
▽ AddTemp f
  TEMP, ← {(5 ÷ 9) × f - 32} f
▽
```

3 Inventory Manager

Warehouse workers add items to an inventory via **AddItem**. The quantity of items is

stored in the shared counter **STOCK**.

```
▽ r←AddItem item
  current←STOCK[item]
  STOCK[item]←current+1
▽
```

4 Weather Station

Each weather station uses the **Record** function to append to shared logs. A dashboard user calls **Report** to show the temperature range and time of latest reading.

```
▽ Record(id temp tm)
  STATION,←id
  TEMPERATURE,←temp
  READAT,←tm
▽

▽ r←Report; lo; hi
  r←([/TEMPERATURE)([/TEMPERATURE)([/READAT)
▽
```

5 Stock Quote Service

A ticker service records a price and a trade count for each symbol. The feed handler calls **Tick** on every trade and client threads call **Quote** at any time to read them back.

```
▽ Tick(sym px)
  :Hold 'PRICES'
    PRICES[sym]←px
    VOLUME[sym]←+1
  :EndHold
▽

▽ r←Quote sym;px;vol
  px←PRICES[sym]
  vol←VOLUME[sym]
  r←px vol
▽
```

6 Key-value Store

A key-value store is accessed via lookups into a quick-access cache. If a key is found by the **Lookup** function (cache hit) it should not wait for a slow write, so two

mutexes are used. If a key is not found, it reads the value from the store and puts it into the cache using **AddEntry** before returning it. **Save** writes to the store and drops the stale cache entry.

```
▽ r←Lookup key
  :Hold 'CACHE'
    :If ~key∈KEYS
      :Hold 'STORE'
        AddEntry key(ReadStore key)
      :EndHold
    :EndIf
    r←CacheValue key
  :EndHold
▽

▽ Save(key val)
  :Hold 'STORE'
    WriteStore key val
  :Hold 'CACHE'
    DropEntry key
  :EndHold
:EndHold
▽
```

7 Tick Report

Many feeds call **Tick** and a dashboard app with many users calls **Report**. They both hold the '**PRICES**' token so access is synchronous. What could go wrong?

```
▽ Tick(sym px)
  :Hold 'PRICES'
    PRICES[sym]←px
  :EndHold
▽

▽ r←Report
  :Hold 'PRICES'
    r←RenderChart PRICES
  :EndHold
▽
```

8 Worker Registry

The scheduler keeps the live worker threads in **WORKERS** so that it can wait for them or stop them. Each worker removes itself when it finishes. Some jobs are very short.

```
▽ Start job;t
  t←Run&job
  WORKERS,←t
```

```

▽
▽ r←Run job
  Process job
  WORKERS~←□TID
  r←0
▽

```

9 Shared Helper Function

In our inventory system, **Receive** takes a list of items and adds each to the inventory using **AddItem**.

```

▽ AddItem item
  :Hold 'STOCK'
    STOCK[item]++1
  :EndHold
▽

▽ Receive items;item
  :Hold 'STOCK'
    :For item :In items
      AddItem item
    :EndFor
  :EndHold
▽

```